

AgroAidApp - Código manual para voz, imagen y branding

Guía para copiar a mano los archivos solicitados: servicios de voz e imagen, botones de microfono/camara, edicion de la pantalla de consulta, perfil y useBranding.

Antes de copiar: instalar dependencias compatibles con Expo usando `npx expo install expo-av expo-image-picker`. `app.json` ya tiene permisos de camara y microfono en el proyecto actual.

Bloque	Archivos
Voz	services/voz.ts, components/MicButton.tsx, app/(tabs)/index.tsx
Imagen	services/imagen.ts, components/CameraButton.tsx, app/(tabs)/index.tsx
Branding	hooks/useBranding.ts, app/(tabs)/perfil.tsx

Bloque voz

Endpoints usados: POST /api/voz/transcribir y POST /api/voz/consulta. El boton llama directamente a /api/voz/consulta y luego la pantalla carga la consulta creada.

AgroAidApp/services/voz.ts - CREAR

```
import { apiFetch } from '@services/api';
import type { ConsultaResponse } from '@services/consultas';

export type TranscripcionResponse = {
  text: string;
};

function createAudioFormData(uri: string, mimeType = 'audio/m4a') {
  const formData = new FormData();
  const extension = mimeType.split('/')[1] ?? 'm4a';

  formData.append('file', {
    uri,
    name: `consulta-voz.${extension}`,
    type: mimeType
  } as unknown as Blob);

  return formData;
}

export function transcribirAudio(uri: string, mimeType = 'audio/m4a') {
  return apiFetch<TranscripcionResponse>('/api/voz/transcribir', {
    method: 'POST',
    body: createAudioFormData(uri, mimeType)
  });
}

export function vozConsulta(uri: string, mimeType = 'audio/m4a') {
  return apiFetch<ConsultaResponse>('/api/voz/consulta', {
    method: 'POST',
    body: createAudioFormData(uri, mimeType)
  });
}
```

AgroAidApp/components/MicButton.tsx - CREAR

```
import { Audio } from 'expo-av';
import { useState } from 'react';
import { ActivityIndicator, Pressable, StyleSheet, Text } from 'react-native';

import { colors } from '@constants/colors';
import type { ConsultaResponse } from '@services/consultas';
import { vozConsulta } from '@services/voz';

type Props = {
```

AgroAidApp - Voz, imagen y branding

```
disabled?: boolean;
onResult: (consulta: ConsultaResponse) => void;
onError?: (message: string) => void;
};

export function MicButton({ disabled, onResult, onError }: Props) {
  const [recording, setRecording] = useState<Audio.Recording | null>(null);
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function startRecording() {
    const permission = await Audio.requestPermissionsAsync();

    if (!permission.granted) {
      onError?.('Permiso de microfono denegado.');
```

```
      return;
    }

    await Audio.setAudioModeAsync({
      allowsRecordingIOS: true,
      playsInSilentModeIOS: true
    });

    const { recording: nextRecording } = await Audio.Recording.createAsync(
      Audio.RecordingOptionsPresets.HIGH_QUALITY
    );

    setRecording(nextRecording);
  }

  async function stopRecording() {
    if (!recording) {
      return;
    }

    setIsSubmitting(true);

    try {
      await recording.stopAndUnloadAsync();
      const uri = recording.getURI();
      setRecording(null);

      if (!uri) {
        onError?.('No se pudo leer el audio grabado.');
```

```
        return;
      }

      const consulta = await vozConsulta(uri, 'audio/m4a');
      onResult(consulta);
    } catch (err) {
      onError?.(
        err instanceof Error ? err.message : 'No se pudo procesar el audio.'
      );
    } finally {
      setIsSubmitting(false);
    }
  }
}
```

AgroAidApp - Voz, imagen y branding

```
    async function handlePress() {
      if (recording) {
        await stopRecording();
        return;
      }

      await startRecording();
    }

    const isDisabled = disabled || isSubmitting;

    return (
      <Pressable
        disabled={isDisabled}
        onPress={handlePress}
        style={[
          styles.button,
          recording && styles.recording,
          isDisabled && styles.disabled
        ]}
      >
        {isSubmitting ? (
          <ActivityIndicator color={colors.text.inverse} />
        ) : (
          <Text style={styles.text}>{recording ? 'Detener voz' : 'Voz'}</Text>
        )}
      </Pressable>
    );
  }
}

const styles = StyleSheet.create({
  button: {
    alignItems: 'center',
    backgroundColor: colors.brand.secondary,
    borderRadius: 8,
    flex: 1,
    justifyContent: 'center',
    minHeight: 44,
    paddingHorizontal: 12
  },
  recording: {
    backgroundColor: colors.risk.critical
  },
  disabled: {
    opacity: 0.55
  },
  text: {
    color: colors.text.inverse,
    fontWeight: '700'
  }
});
```

Bloque imagen

Endpoint usado: POST /api/consulta/imagen. El backend devuelve etiqueta_parseada, consulta_generada y evaluacion.

AgroAidApp/services/imagen.ts - CREAR

```
import { apiFetch } from '@services/api';
import type { ConsultaResponse } from '@services/consultas';

export type EtiquetaParseada = {
  producto?: string | null;
  principio_activo?: string | null;
  cultivos_o_especies_permitidos?: string[] | null;
  dosis_recomendada?: string | null;
  intervalo_carencia_dias?: number | null;
  epp_requerido?: string[] | null;
  categoria_toxicologica?: string | null;
  frases_de_seguridad?: string[] | null;
  texto_completo_visible?: string | null;
  [key: string]: unknown;
};

export type ImagenConsultaResponse = {
  etiqueta_parseada: EtiquetaParseada;
  consulta_generada: string;
  evaluacion: ConsultaResponse;
};

function createImageFormData(uri: string, mimeType = 'image/jpeg') {
  const formData = new FormData();
  const extension = mimeType.split('/')[1] ?? 'jpg';

  formData.append('file', {
    uri,
    name: `etiqueta.${extension}`,
    type: mimeType
  } as unknown as Blob);

  return formData;
}

export function analizarEtiqueta(uri: string, mimeType = 'image/jpeg') {
  return apiFetch<ImagenConsultaResponse>('/api/consulta/imagen', {
    method: 'POST',
    body: createImageFormData(uri, mimeType)
  });
}
```

AgroAidApp/components/CameraButton.tsx - CREAR

```
import * as ImagePicker from 'expo-image-picker';
```

AgroAidApp - Voz, imagen y branding

```
import { useState } from 'react';
import { ActivityIndicator, Pressable, StyleSheet, Text } from 'react-native';

import { colors } from '@constants/colors';
import { analizarEtiqueta, type ImagenConsultaResponse } from '@services/imagen';

type Props = {
  disabled?: boolean;
  onResult: (result: ImagenConsultaResponse) => void;
  onError?: (message: string) => void;
};

export function CameraButton({ disabled, onResult, onError }: Props) {
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function handlePress() {
    const permission = await ImagePicker.requestCameraPermissionsAsync();

    if (!permission.granted) {
      onError?.('Permiso de camara denegado.');
```

AgroAidApp - Voz, imagen y branding

```
const isDisabled = disabled || isSubmitting;

return (
  <Pressable
    disabled={isDisabled}
    onPress={handlePress}
    style={[styles.button, isDisabled && styles.disabled]}
  >
    {isSubmitting ? (
      <ActivityIndicator color={colors.text.inverse} />
    ) : (
      <Text style={styles.text}>Camara</Text>
    )}
  </Pressable>
);
}

const styles = StyleSheet.create({
  button: {
    alignItems: 'center',
    backgroundColor: colors.brand.accent,
    borderRadius: 8,
    flex: 1,
    justifyContent: 'center',
    minHeight: 44,
    paddingHorizontal: 12
  },
  disabled: {
    opacity: 0.55
  },
  text: {
    color: colors.text.inverse,
    fontWeight: '700'
  }
});
```

Pantalla de consulta

Reemplazar el contenido completo de app/(tabs)/index.tsx. Conserva consulta por texto, agrega voz y camara, y recarga la sesion creada por backend.

AgroAidApp/app/(tabs)/index.tsx - EDITAR - reemplazar contenido completo

```
import { useState } from 'react';
import {
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View
} from 'react-native';

import { CameraButton } from '@components/CameraButton';
import { ConsultaCard } from '@components/ConsultaCard';
import { LoadingOverlay } from '@components/LoadingOverlay';
import { MicButton } from '@components/MicButton';
import { RespuestaInput } from '@components/RespuestaInput';
import { colors } from '@constants/colors';
import { useConsulta } from '@hooks/useConsulta';
import { useLocation } from '@hooks/useLocation';
import type { ImagenConsultaResponse } from '@services/imagen';
import type { ConsultaResponse } from '@services/consultas';

export default function ConsultaTab() {
  const [texto, setTexto] = useState('');
  const [respuesta, setRespuesta] = useState('');
  const [mediaError, setMediaError] = useState<string | null>(null);
  const { consulta, error, isLoading, iniciar, responder, reset, cargar } =
    useConsulta();
  const {
    coords,
    error: locationError,
    isLoading: isLocating,
    requestLocation
  } = useLocation();

  async function handleIniciar() {
    if (!texto.trim()) {
      return;
    }

    await iniciar({
      consulta_inicial: texto.trim(),
      lat: coords?.lat,
      lon: coords?.lon
    });
  }

  async function handleResponder() {
```

```
    if (!respuesta.trim()) {
        return;
    }

    await responder(respuesta.trim());
    setRespuesta('');
}

async function handleVozResult(result: ConsultaResponse) {
    setMediaError(null);
    await cargar(result.consulta_id);
}

async function handleImagenResult(result: ImagenConsultaResponse) {
    setMediaError(null);
    setTexto(result.consulta_generada);
    await cargar(result.evaluacion.consulta_id);
}

function handleMediaError(message: string) {
    setMediaError(message);
}

return (
    <View style={styles.container}>
        <LoadingOverlay visible={isLoading || isLocating} />
        <ScrollView contentContainerStyle={styles.screen}>
            <Text style={styles.title}>Consulta agronomica</Text>

            {!consulta ? (
                <View style={styles.startPanel}>
                    <TextInput
                        multiline
                        onChangeText={setTexto}
                        placeholder="Describe el problema, cultivo, producto o riesgo..."
                        placeholderTextColor={colors.text.secondary}
                        style={styles.input}
                        value={texto}
                    />

                    <View style={styles.mediaRow}>
                        <MicButton
                            disabled={isLoading}
                            onError={handleMediaError}
                            onResult={handleVozResult}
                        />
                        <CameraButton
                            disabled={isLoading}
                            onError={handleMediaError}
                            onResult={handleImagenResult}
                        />
                    </View>

                    <Pressable onPress={requestLocation} style={styles.secondaryButton}>
                        <Text style={styles.secondaryButtonText}>
                            {coords ? 'Ubicacion agregada' : 'Agregar ubicacion'}
                        </Text>
                    </Pressable>
                </View>
            ) : (
                <View style={styles.resultPanel}>
                    <Text style={styles.resultText}>{result.consulta_generada}</Text>
                    <Image style={styles.resultImage} src={result.imagen_consulta} alt="Imagen de la consulta generada" data-bbox="115 885 285 901" />
                </View>
            )}
        </ScrollView>
    </View>
)
```

```

        </Text>
    </Pressable>

    <Pressable
        disabled={isLoading || !texto.trim()}
        onPress={handleIniciar}
        style={[
            styles.button,
            (isLoading || !texto.trim()) && styles.disabled
        ]}
    >
        <Text style={styles.buttonText}>Iniciar consulta</Text>
    </Pressable>
</View>
) : (
    <View style={styles.flow}>
        <ConsultaCard consulta={consulta} />

        {!consulta.completado ? (
            <RespuestaInput
                disabled={isLoading}
                onChangeText={setRespuesta}
                onSubmit={handleResponder}
                value={respuesta}
            />
        ) : null}

        <Pressable onPress={reset} style={styles.secondaryButton}>
            <Text style={styles.secondaryButtonText}>Nueva consulta</Text>
        </Pressable>
    </View>
)}

    {locationError ? <Text style={styles.warning}>{locationError}</Text> : null}
    {mediaError ? <Text style={styles.error}>{mediaError}</Text> : null}
    {error ? <Text style={styles.error}>{error}</Text> : null}
</ScrollView>
</View>
);
}

const styles = StyleSheet.create({
    container: {
        flex: 1
    },
    screen: {
        backgroundColor: colors.surface.background,
        flexGrow: 1,
        gap: 14,
        padding: 20
    },
    title: {
        color: colors.text.primary,
        fontSize: 26,
        fontWeight: '700'
    },
},

```

```
startPanel: {
  gap: 12
},
flow: {
  gap: 14
},
input: {
  backgroundColor: colors.surface.card,
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  color: colors.text.primary,
  minHeight: 150,
  padding: 14,
  textAlignVertical: 'top'
},
mediaRow: {
  flexDirection: 'row',
  gap: 10
},
button: {
  alignItems: 'center',
  backgroundColor: colors.brand.primary,
  borderRadius: 8,
  justifyContent: 'center',
  minHeight: 48
},
disabled: {
  opacity: 0.55
},
buttonText: {
  color: colors.text.inverse,
  fontWeight: '700'
},
secondaryButton: {
  alignItems: 'center',
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  justifyContent: 'center',
  minHeight: 44
},
secondaryButtonText: {
  color: colors.brand.primary,
  fontWeight: '700'
},
warning: {
  color: colors.risk.medium
},
error: {
  color: colors.risk.critical
}
});
```

Branding y perfil

Completa useBranding con tenantSlug editable y expone colores actuales. Perfil agrega logo remoto, input de tenant y boton para recargar branding.

AgroAidApp/hooks/useBranding.ts - EDITAR/COMPLETAR - reemplazar contenido completo

```
import {
  createContext,
  createElement,
  type ReactNode,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState
} from 'react';

import { colors } from '@constants/colors';
import { apiFetch } from '@services/api';

export type Branding = {
  logo_url: string | null;
  primary_color: string;
  accent_color: string;
  app_name: string;
  footer_text: string;
};

const defaultBranding: Branding = {
  logo_url: null,
  primary_color: colors.brand.primary,
  accent_color: colors.brand.secondary,
  app_name: 'AgroAid',
  footer_text: 'AgroAid - Seguridad agricola integral'
};

type BrandingContextValue = {
  branding: Branding;
  tenantSlug: string;
  isLoading: boolean;
  primaryColor: string;
  accentColor: string;
  setTenantSlug: (slug: string) => void;
  reloadBranding: (slug?: string) => Promise<void>;
};

const BrandingContext = createContext<BrandingContextValue | null>(null);

function normalizeBranding(data: Partial<Branding>): Branding {
  return {
    logo_url: data.logo_url ?? defaultBranding.logo_url,
    primary_color: data.primary_color ?? defaultBranding.primary_color,
    accent_color: data.accent_color ?? defaultBranding.accent_color,
```

AgroAidApp - Voz, imagen y branding

```
    app_name: data.app_name ?? defaultBranding.app_name,
    footer_text: data.footer_text ?? defaultBranding.footer_text
  };
}

export function BrandingProvider({ children }: { children: ReactNode }) {
  const [branding, setBranding] = useState<Branding>(defaultBranding);
  const [tenantSlug, setTenantSlug] = useState('default');
  const [isLoading, setIsLoading] = useState(true);

  const reloadBranding = useCallback(
    async (slug = tenantSlug) => {
      const normalizedSlug = slug.trim() || 'default';
      setTenantSlug(normalizedSlug);
      setIsLoading(true);

      try {
        const params = new URLSearchParams({ slug: normalizedSlug });
        const data = await apiFetch<Branding>(`/api/branding?${params}`, {
          skipAuth: true
        });
        setBranding(normalizeBranding(data));
      } catch {
        setBranding(defaultBranding);
      } finally {
        setIsLoading(false);
      }
    },
    [tenantSlug]
  );

  useEffect(() => {
    reloadBranding('default');
  }, []);

  const value = useMemo<BrandingContextValue>(
    () => ({
      branding,
      tenantSlug,
      isLoading,
      primaryColor: branding.primary_color,
      accentColor: branding.accent_color,
      setTenantSlug,
      reloadBranding
    }),
    [branding, tenantSlug, isLoading, reloadBranding]
  );

  return createElement(BrandingContext.Provider, { value }, children);
}

export function useBranding() {
  const context = useContext(BrandingContext);

  if (!context) {
    throw new Error('useBranding debe usarse dentro de BrandingProvider');
  }
}
```

```
    }  
    return context;  
  }  
}
```

AgroAidApp/app/(tabs)/perfil.tsx - EDITAR - reemplazar contenido completo

```
import { router } from 'expo-router';  
import { Image, Pressable, StyleSheet, Text, TextInput, View } from 'react-native';  
  
import { colors } from '@constants/colors';  
import { useAuth } from '@hooks/useAuth';  
import { useBranding } from '@hooks/useBranding';  
  
export default function PerfilTab() {  
  const { isAuthenticated, logout } = useAuth();  
  const {  
    branding,  
    tenantSlug,  
    isLoading,  
    setTenantSlug,  
    reloadBranding  
  } = useBranding();  
  
  async function handleLogout() {  
    await logout();  
    router.replace('/login');  
  }  
  
  return (  
    <View style={styles.screen}>  
      <Text style={styles.title}>Perfil</Text>  
  
      <View style={styles.card}>  
        {branding.logo_url ? (  
          <Image source={{ uri: branding.logo_url }} style={styles.logo} />  
        ) : null}  
        <Text style={styles.label}>Aplicacion</Text>  
        <Text style={styles.value}>{branding.app_name}</Text>  
        <Text style={styles.footer}>{branding.footer_text}</Text>  
      </View>  
  
      <View style={styles.card}>  
        <Text style={styles.label}>Tenant</Text>  
        <TextInput  
          autoCapitalize="none"  
          onChangeText={setTenantSlug}  
          placeholder="default"  
          placeholderTextColor={colors.text.secondary}  
          style={styles.input}  
          value={tenantSlug}  
        />  
        <Pressable  
          disabled={isLoading}  
          onPress={() => reloadBranding(tenantSlug)}  
        />  
      </View>  
    </View>  
  );  
}
```

```
        style={styles.secondaryButton, isLoading && styles.disabled}}
      >
        <Text style={styles.secondaryButtonText}>Actualizar branding</Text>
      </Pressable>
    </View>

    <View style={styles.card}>
      <Text style={styles.label}>Sesion</Text>
      <Text style={styles.value}>
        {isLoading ? 'Sesion activa' : 'Sin sesion'}
      </Text>
    </View>

    <Pressable onPress={handleLogout} style={styles.button}>
      <Text style={styles.buttonText}>Cerrar sesion</Text>
    </Pressable>
  </View>
);
}
```

```
const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flex: 1,
    gap: 14,
    padding: 20
  },
  title: {
    color: colors.text.primary,
    fontSize: 24,
    fontWeight: '700'
  },
  card: {
    backgroundColor: colors.surface.card,
    borderColor: colors.surface.border,
    borderRadius: 8,
    borderWidth: 1,
    gap: 8,
    padding: 14
  },
  logo: {
    alignSelf: 'flex-start',
    height: 54,
    resizeMode: 'contain',
    width: 160
  },
  label: {
    color: colors.text.secondary,
    fontSize: 13,
    fontWeight: '700',
    textTransform: 'uppercase'
  },
  value: {
    color: colors.text.primary,
    fontSize: 17,
    fontWeight: '700'
  }
});
```

```
    },
    footer: {
      color: colors.text.secondary,
      lineHeight: 20
    },
    input: {
      borderColor: colors.surface.border,
      borderRadius: 8,
      borderWidth: 1,
      color: colors.text.primary,
      minHeight: 44,
      paddingHorizontal: 12
    },
    secondaryButton: {
      alignItems: 'center',
      borderColor: colors.surface.border,
      borderRadius: 8,
      borderWidth: 1,
      justifyContent: 'center',
      minHeight: 44
    },
    secondaryButtonText: {
      color: colors.brand.primary,
      fontWeight: '700'
    },
    disabled: {
      opacity: 0.55
    },
    button: {
      alignItems: 'center',
      backgroundColor: colors.risk.critical,
      borderRadius: 8,
      justifyContent: 'center',
      minHeight: 48
    },
    buttonText: {
      color: colors.text.inverse,
      fontSize: 16,
      fontWeight: '700'
    }
  }
});
```

Checklist final

- Crear services/voz.ts y services/imagen.ts.
- Crear components/MicButton.tsx y components/CameraButton.tsx.
- Reemplazar app/(tabs)/index.tsx, app/(tabs)/perfil.tsx y hooks/useBranding.ts.
- Instalar dependencias: npx expo install expo-av expo-image-picker.
- Probar login primero: los endpoints de voz e imagen requieren token.
- Probar microfono y camara en dispositivo o emulador con permisos concedidos.